
Estructura de Exposición

Fundamentos del Paralelismo y Rendimiento

Curso: Computación Paralela y Distribuida
Código: SIS225
Ciclo: VII
Semestre: 2026-II
Docente: Dr. Ing. Robert Antonio Romero Flores
Unidad: Unidad 1 – Semanas 1 y 2
Duración: 16 minutos (4 min por persona)

Integrantes:

1. Integrante 1 – Concurrencia
2. Integrante 2 – Paralelismo
3. Integrante 3 – Rendimiento y consistencia
4. Integrante 4 – Ejemplos en Python

15 de septiembre de 2026

Índice

1. Información General de la Exposición	2
2. PERSONA 1: ¿Qué es concurrencia?	2
2.1. Objetivo de la intervención	2
2.2. Contenido teórico	2
2.2.1. Definición de concurrencia	2
2.2.2. Concurrencia vs. secuencialidad	3
2.2.3. Ejemplo cotidiano	3
2.2.4. Concurrencia en computación	3
2.2.5. Modelos de concurrencia	3
2.3. Preguntas que debe poder responder	4
2.4. Estructura sugerida de la intervención (4 min)	4
3. PERSONA 2: ¿Qué es paralelismo?	4
3.1. Objetivo de la intervención	4
3.2. Contenido teórico	4
3.2.1. Definición de paralelismo	4
3.2.2. Diferencia clave: concurrencia vs. paralelismo	4
3.2.3. Tipos de paralelismo	5
3.2.4. Arquitecturas paralelas	5
3.3. Preguntas que debe poder responder	5
3.4. Estructura sugerida de la intervención (4 min)	5
4. PERSONA 3: Rendimiento y consistencia	6
4.1. Objetivo de la intervención	6
4.2. Contenido teórico	6
4.2.1. Métricas de rendimiento	6
4.2.2. Ejemplo numérico	6
4.2.3. Ley de Amdahl	7
4.2.4. Consistencia	7
4.3. Preguntas que debe poder responder	7
4.4. Estructura sugerida de la intervención (4 min)	7
5. PERSONA 4: Ejemplos en Python	8
5.1. Objetivo de la intervención	8
5.2. Contenido teórico	8
5.2.1. Diferencia entre threading y multiprocessing	8
5.2.2. Ejemplo 1: Concurrencia con threading	8
5.2.3. Ejemplo 2: Paralelismo con multiprocessing	8
5.2.4. Ejemplo 3: Cálculo de speedup y eficiencia	9
5.2.5. Cierre de la exposición	9
5.3. Preguntas que debe poder responder	10
5.4. Estructura sugerida de la intervención (4 min)	10
6. Resumen de Fórmulas Clave	10
7. Glosario	11
8. Checklist por Persona	11
9. Consejos para la Exposición	12

1. Información General de la Exposición

Datos de la exposición

- **Tema:** Fundamentos del paralelismo y rendimiento.
- **Semanas del sílabo:** Semana 1 y Semana 2 (Unidad 1).
- **Duración total:** 16 minutos.
- **Integrantes:** 4 personas.
- **Tiempo por persona:** 4 minutos.
- **Objetivo:** Comprender la diferencia entre concurrencia y paralelismo, y cómo se mide el rendimiento en sistemas paralelos.

Semanas del sílabo involucradas

azulUP	
1	Fundamentos del paralelismo. Rendimiento y consistencia.
2	Rendimiento y consistencia.

Estructura general

azulUP			
1	Persona 1	¿Qué es concurrencia?	4 min
2	Persona 2	¿Qué es paralelismo?	4 min
3	Persona 3	Rendimiento y consistencia	4 min
4	Persona 4	Ejemplos en Python	4 min

2. PERSONA 1: ¿Qué es concurrencia?

2.1 Objetivo de la intervención

Explicar qué es la concurrencia, cómo se diferencia de la secuencialidad y por qué es importante en sistemas computacionales.

2.2 Contenido teórico

2.2.1 Definición de concurrencia

Definición

La **concurrencia** es la capacidad de un sistema para ejecutar múltiples tareas **al mismo tiempo lógico**, aunque no necesariamente en el mismo instante físico.

Idea clave: las tareas progresan de manera intercalada, compartiendo recursos como el CPU.

2.2.2 Concurrencia vs. secuencialidad

azulClaroazulUP		
azulClaroEjecución	Una tarea tras otra.	Varias tareas intercaladas.
azulClaroRecursos	Un solo recurso a la vez.	Recursos compartidos.
azulClaroEjemplo	Un solo hilo ejecutando pasos.	Varios hilos turnándose el CPU.
azulClaroVentaja	Simplicidad.	Mejor uso del CPU.

2.2.3 Ejemplo cotidiano

Ejemplo para exponer

Imagina que estás cocinando y, mientras hierves pasta, cortas verduras y revisas el horno. No haces las tres cosas exactamente al mismo tiempo, pero **avanzas en todas** sin esperar a que una termine para empezar la otra.

Eso es concurrencia: **progreso intercalado de tareas**.

2.2.4 Concurrencia en computación

¿Dónde aparece la concurrencia?

- Sistemas operativos: múltiples procesos compitiendo por el CPU.
- Servidores web: atender varias peticiones a la vez.
- Aplicaciones de escritorio: interfaz gráfica mientras se procesa algo en segundo plano.
- Hilos (threads): unidades de ejecución dentro de un proceso.

2.2.5 Modelos de concurrencia

azulClaroazulUP	
azulClaroHilos (threads)	Comparten memoria dentro de un proceso.
azulClaroProcesos	Memoria independiente, comunicación por IPC.
azulClaroActores	Cada actor tiene su propio estado y se comunican por mensajes.
azulClaroReactivo	Responde a eventos de forma asíncrona.

2.3 Preguntas que debe poder responder

Preguntas clave

1. ¿Qué es la concurrencia?
2. ¿En qué se diferencia de la ejecución secuencial?
3. ¿Qué es un hilo y en qué se diferencia de un proceso?
4. ¿Por qué la concurrencia mejora el uso del CPU?

2.4 Estructura sugerida de la intervención (4 min)

1. (30 s) Presentación del tema y objetivo.
2. (1 min) Definición de concurrencia y comparación con secuencialidad.
3. (1 min) Ejemplo cotidiano (cocina).
4. (1 min) Concurrencia en computación: hilos, procesos, servidores.
5. (30 s) Transición: “Ahora que sabemos qué es concurrencia, veamos qué es paralelismo”.

3. PERSONA 2: ¿Qué es paralelismo?

3.1 Objetivo de la intervención

Explicar qué es el paralelismo, cómo se diferencia de la concurrencia y qué tipos existen.

3.2 Contenido teórico

3.2.1 Definición de paralelismo

Definición

El **paralelismo** es la ejecución **simultánea física** de múltiples tareas en diferentes unidades de procesamiento (núcleos, procesadores, nodos).

Idea clave: las tareas se ejecutan **exactamente al mismo tiempo**, no intercaladas.

3.2.2 Diferencia clave: concurrencia vs. paralelismo

La diferencia más importante de la exposición

- **Concurrencia:** varias tareas progresan intercaladamente. Puede ocurrir en un solo núcleo.
- **Paralelismo:** varias tareas se ejecutan simultáneamente. Requiere múltiples núcleos.

Regla: todo paralelismo implica concurrencia, pero no toda concurrencia implica paralelismo.

azulClaroazulUP		
azulClaroEjecución	Intercalada.	Simultánea.
azulClaroNúcleos	Puede ser 1.	Requiere 2 o más.
azulClaroObjetivo	Mejorar uso del CPU.	Reducir tiempo de ejecución.
azulClaroEjemplo	Hilos turnándose el CPU.	4 núcleos ejecutando 4 tareas.

3.2.3 Tipos de paralelismo

Tipos de paralelismo

Paralelismo de datos: se dividen los datos y cada núcleo procesa una parte. Ejemplo: sumar dos vectores.

Paralelismo de tareas: se dividen las operaciones en tareas independientes. Ejemplo: pipeline de procesamiento.

Paralelismo de bits: se procesan varios bits a la vez (histórico).

Paralelismo de instrucciones: el hardware ejecuta varias instrucciones simultáneamente (ILP).

3.2.4 Arquitecturas paralelas

azulClaroazulUP	
azulClaroSISD	Un solo procesador, una sola instrucción, un solo dato.
azulClaroSIMD	Múltiples procesadores, misma instrucción, múltiples datos.
azulClaroMIMD	Múltiples procesadores, distintas instrucciones, distintos datos.

3.3 Preguntas que debe poder responder

Preguntas clave

1. ¿Qué es el paralelismo?
2. ¿Cuál es la diferencia entre concurrencia y paralelismo?
3. ¿Qué tipos de paralelismo existen?
4. ¿Qué diferencia hay entre SIMD y MIMD?

3.4 Estructura sugerida de la intervención (4 min)

1. (30 s) Definición de paralelismo.
2. (1 min) Diferencia clave concurrencia vs. paralelismo (con tabla).
3. (1 min) Tipos de paralelismo (datos, tareas, bits, instrucciones).
4. (1 min) Arquitecturas paralelas: SISD, SIMD, MIMD.

5. **(30 s)** Transición: “Ahora que sabemos qué es paralelismo, veamos cómo medir su rendimiento”.

4. PERSONA 3: Rendimiento y consistencia

4.1 Objetivo de la intervención

Explicar cómo se mide el rendimiento en sistemas paralelos (speedup, eficiencia, Ley de Amdahl) y qué es la consistencia.

4.2 Contenido teórico

4.2.1 Métricas de rendimiento

Métricas fundamentales

Tiempo secuencial (T_{seq}): tiempo de ejecución con 1 procesador.

Tiempo paralelo (T_{par}): tiempo de ejecución con p procesadores.

Speedup (S): $S = \frac{T_{seq}}{T_{par}}$.

Eficiencia (E): $E = \frac{S}{p}$.

4.2.2 Ejemplo numérico

Ejemplo para exponer

Un programa tarda:

- 100 segundos en secuencial ($T_{seq} = 100$)
- 30 segundos con 4 procesadores ($T_{par} = 30, p = 4$)

$$S = \frac{100}{30} = 3,33$$

$$E = \frac{3,33}{4} = 0,83 \text{ (83 \%)}$$

Interpretación: se obtuvo un speedup de 3.33x con 4 procesadores, con una eficiencia del 83 %. El 17 % restante se perdió en overhead.

4.2.3 Ley de Amdahl

Ley de Amdahl

Establece el speedup máximo alcanzable cuando solo una fracción f del programa es paralelizable:

$$S_{max} = \frac{1}{(1-f) + \frac{f}{p}}$$

Ejemplo: si $f = 0,8$ y $p = 4$:

$$S_{max} = \frac{1}{0,2 + \frac{0,8}{4}} = \frac{1}{0,4} = 2,5$$

Aunque tengas 4 procesadores, el speedup máximo es 2.5 por el 20 % secuencial.

4.2.4 Consistencia

¿Qué es la consistencia?

La **consistencia** se refiere a cómo se garantiza que todos los procesadores vean el mismo estado de la memoria compartida.

Modelos de consistencia:

- **Consistencia estricta:** cualquier lectura ve la última escritura.
- **Consistencia secuencial:** las operaciones se ven en algún orden secuencial.
- **Consistencia relajada:** se permiten reordenamientos para mejorar el rendimiento.

4.3 Preguntas que debe poder responder

Preguntas clave

1. ¿Cómo se calcula el speedup?
2. ¿Qué es la eficiencia?
3. ¿Qué dice la Ley de Amdahl?
4. ¿Qué es la consistencia de memoria?
5. ¿Por qué no se puede lograr speedup infinito?

4.4 Estructura sugerida de la intervención (4 min)

1. **(30 s)** Introducción a las métricas de rendimiento.
2. **(1 min)** Speedup y eficiencia con ejemplo numérico.
3. **(1 min)** Ley de Amdahl con ejemplo.
4. **(1 min)** Consistencia de memoria y sus modelos.
5. **(30 s)** Transición: “Ahora veamos ejemplos prácticos en Python”.

5. PERSONA 4: Ejemplos en Python

5.1 Objetivo de la intervención

Mostrar ejemplos prácticos de concurrencia y paralelismo en Python, y cerrar la exposición.

5.2 Contenido teórico

5.2.1 Diferencia entre threading y multiprocessing

Threading vs. Multiprocessing

- **threading:** concurrencia dentro de un proceso. Comparten memoria. Limitado por el GIL en Python.
- **multiprocessing:** paralelismo real con múltiples procesos. Memoria independiente. Sin GIL.

5.2.2 Ejemplo 1: Concurrencia con threading

```
1 import threading
2 import time
3
4 def tarea(nombre, duracion):
5     print(f"Iniciando {nombre}")
6     time.sleep(duracion)
7     print(f"Terminando {nombre}")
8
9 # Crear hilos
10 hilo1 = threading.Thread(target=tarea, args=("Tarea 1", 2))
11 hilo2 = threading.Thread(target=tarea, args=("Tarea 2", 2))
12
13 inicio = time.time()
14 hilo1.start()
15 hilo2.start()
16 hilo1.join()
17 hilo2.join()
18 fin = time.time()
19
20 print(f"Tiempo total: {fin - inicio:.2f} segundos")
21 # Ambas tareas tardan 2 segundos, pero el total es ~2s
22 # porque se ejecutan concurrentemente
```

Listing 1: Concurrencia con threading

5.2.3 Ejemplo 2: Paralelismo con multiprocessing

```
1 from multiprocessing import Pool
2 import time
3
4 def tarea_pesada(n):
5     """Simula una tarea costosa."""
6     return sum(i*i for i in range(n))
7
8 if __name__ == "__main__":
9     datos = [10**6, 10**6, 10**6, 10**6]
10
```

```

11  # Secuencial
12  inicio = time.time()
13  resultados_seq = [tarea_pesada(n) for n in datos]
14  tiempo_seq = time.time() - inicio
15  print(f"Secuencial: {tiempo_seq:.2f}s")
16
17  # Paralelo
18  inicio = time.time()
19  with Pool(processes=4) as pool:
20      resultados_par = pool.map(tarea_pesada, datos)
21  tiempo_par = time.time() - inicio
22  print(f"Paralelo: {tiempo_par:.2f}s")
23
24  # Speedup
25  speedup = tiempo_seq / tiempo_par
26  print(f"Speedup: {speedup:.2f}x")

```

Listing 2: Paralelismo con multiprocessing

5.2.4 Ejemplo 3: Cálculo de speedup y eficiencia

```

1  def calcular_metricas(t_seq, t_par, p):
2      speedup = t_seq / t_par
3      eficiencia = speedup / p
4      print(f"Speedup: {speedup:.2f}x")
5      print(f"Eficiencia: {eficiencia:.2%}")
6      return speedup, eficiencia
7
8  # Ejemplo
9  calcular_metricas(t_seq=100, t_par=30, p=4)
10 # Speedup: 3.33x
11 # Eficiencia: 83.33%

```

Listing 3: Cálculo de speedup y eficiencia

5.2.5 Cierre de la exposición

Conclusiones finales

- **Concurrencia:** varias tareas progresan intercaladamente.
- **Paralelismo:** varias tareas se ejecutan simultáneamente.
- **Rendimiento:** se mide con speedup y eficiencia.
- **Ley de Amdahl:** el speedup tiene límites por la parte secuencial.
- **Python:** ofrece threading (concurrencia) y multiprocessing (paralelismo).

5.3 Preguntas que debe poder responder

Preguntas clave

1. ¿Cuándo usar threading y cuándo multiprocessing?
2. ¿Qué es el GIL y cómo afecta?
3. ¿Cómo se mide el speedup en Python?
4. ¿Qué conclusiones sacamos del tema?

5.4 Estructura sugerida de la intervención (4 min)

1. **(30 s)** Diferencia threading vs. multiprocessing.
2. **(1 min)** Ejemplo 1: threading (conurrencia).
3. **(1 min)** Ejemplo 2: multiprocessing (paralelismo).
4. **(1 min)** Ejemplo 3: cálculo de speedup.
5. **(30 s)** Conclusiones y cierre.

6. Resumen de Fórmulas Clave

azulClaroazulUP	
azulClaroSpeedup	$S = \frac{T_{seq}}{T_{par}}$
azulClaroEficiencia	$E = \frac{S}{p}$
azulClaroLey de Amdahl	$S_{max} = \frac{1}{(1 - f) + \frac{f}{p}}$
azulClaroOverhead	$T_{overhead} = T_{par} - \frac{T_{seq}}{p}$

7. Glosario

azulClaroazulUP	
azulClaroConcurrencia	Varias tareas progresan intercaladamente.
azulClaroParalelismo	Varias tareas se ejecutan simultáneamente.
azulClaroHilo (thread)	Unidad de ejecución dentro de un proceso.
azulClaroProceso	Programa en ejecución con memoria independiente.
azulClaroSpeedup	Aceleración obtenida.
azulClaroEficiencia	Speedup por procesador.
azulClaroLey de Amdahl	Límite del speedup por la parte secuencial.
azulClaroConsistencia	Garantía de que todos ven el mismo estado de memoria.
azulClaroGIL	Global Interpreter Lock de Python.
azulClaroOverhead	Costo adicional por coordinación.

8. Checklist por Persona

Checklist Persona 1 – Concurrencia

- ☐ Definir concurrencia.
- ☐ Comparar con secuencialidad.
- ☐ Explicar ejemplo de la cocina.
- ☐ Mencionar hilos y procesos.

Checklist Persona 2 – Paralelismo

- ☐ Definir paralelismo.
- ☐ Explicar diferencia con concurrencia.
- ☐ Mencionar tipos de paralelismo.
- ☐ Explicar SISD, SIMD, MIMD.

Checklist Persona 3 – Rendimiento

- ☐ Explicar speedup y eficiencia.
- ☐ Resolver ejemplo numérico.
- ☐ Explicar Ley de Amdahl.
- ☐ Explicar consistencia de memoria.

Checklist Persona 4 – Ejemplos en Python

- ☐ Diferenciar threading y multiprocessing.
- ☐ Dominar el ejemplo de threading.
- ☐ Dominar el ejemplo de multiprocessing.
- ☐ Explicar el cálculo de speedup.
- ☐ Cerrar con conclusiones.

9. Consejos para la Exposición

Recomendaciones

1. **Ensayo cronometrado:** cada persona debe practicar su parte en 4 minutos.
2. **Transiciones claras:** cada uno debe enlazar con el siguiente.
3. **Ejemplo común:** usar el mismo caso (procesar datos) en las 4 intervenciones.
4. **Diapositivas visuales:** usar diagramas, tablas y código resaltado.
5. **Anticipar preguntas:** preparar respuestas a las preguntas clave.
6. **Cierre conjunto:** los 4 integrantes al frente para conclusiones.

Errores que se deben evitar

- Leer diapositivas textualmente.
- No coordinarse entre integrantes.
- Exceder el tiempo asignado.
- No dar ejemplos concretos.
- No conectar con el proyecto final.